Solver-In-The-Loop Training with Multistep Inputs for Capturing Fine-Scale Turbulence

Amelie Lam; Héloïse Méheut; Sara El Bouch; André Ferrari

Université Côte d'Azur, Observatoire de la Côte d'Azur, CNRS, Laboratoire Lagrange, Nice, France

Keywords: 2D Turbulence, Differentiable Physics, Solver-In-The-Loop, Deep Learning

Abstract. Understanding turbulence is crucial for studying gas-dust dynamics in planet forming disks, yet resolving small-scale structures remains computationally expensive. To address this challenge, we employ a hybrid simulation framework that couples a coarse resolution fluid solver with a convolutional neural network to model unresolved dynamics during time integration. The network operates on solver outputs and is trained in a solver-in-the-loop setting, in which a differentiable fluid solver is integrated directly into optimisation, enabling end-to-end training against downsampled high-resolution direct numerical simulations. To improve long-term predictive performance, the model is trained on extended temporal rollouts. The framework has previously been evaluated on 2D homogeneous isotropic turbulence, where training on extended rollouts significantly improved long-term accuracy. Our main contribution is the introduction of a multi-step input framework in which the neural network receives both the post-solver state and velocity fields from previous timesteps. We show that this modification allows the model to be trained with shorter rollout lengths while surpassing the baseline model in terms of mean squared error. We further demonstrate that this modification, not only enables faster computation time and memory usage, but also preserves the ability to reproduce key statistical properties of the turbulence.

1 Introduction

Planet-forming disks are rotating thin disks around young stars, composed primarily of gas and a small fraction of dust particles that eventually coagulate and grow to form planets. Observations indicate that these disks are turbulent [1, 2], and this assumption is commonly incorporated into disk models, where turbulence drives angular momentum transport [3] and/or particle diffusion [4, 5]. Provided that these protoplanetary disks span astronomical length scales, direct resolution of all turbulent scales is computationally prohibitive. As a result, disk models rely on approximations. In particular, one often models disks in 2D, while local dynamics are often studied using a small box compared to the disk size [6]. However, integrating the complex behaviours captured in these local simulations into larger scale models remains challenging. Attempts to bridge this gap include implicit large eddy simulations (ILESs) [7], yet convergence to direct numerical simulations (DNSs) still requires high resolution. These results highlight the need for better models of small scale turbulence.

In parallel, data-driven methods have emerged as a promising approach for modelling fluid systems at reduced computational cost. However, turbulence modelling remains challenging due to its inherently multiscale nature, and existing approaches often struggle with generalisation and long-term stability. To address this, a growing body of work leverages differentiable physics frameworks [8] to incorporate physical structure into learning-based models [9–12], leading to improved accuracy and stability. Nevertheless, these approaches remain computationally expensive, particularly for 2D and 3D settings, due to the cost of gradient backpropagation through long rollout trajectories.

In this work, we build on prior approaches based on differentiable physics to develop a hybrid framework that combines machine learning with established numerical methods. Our goal is to construct efficient, data-driven turbulence models for planet-forming disks. While previous studies often used only the solver output as input of the neural network, we additionally provide velocity fields from previous timesteps. This provides more comprehensive temporal and dynamical information, enabling the use of shorter

rollout lengths during training while maintaining performance comparable to baseline models, and thus improving training efficiency.

Within this paper, section 2 presents the methodology used to construct our framework, section 3 discusses on the result obtained, and in section 4 we summarise our finding.

2 Methodology

2.1 Background

Focusing on the small scales of a planet-forming disk, we assume the flow to be incompressible [13]. The flow is governed by the Navier-Stokes equations as follows,

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \frac{1}{Re}\nabla^2 \mathbf{u} + \mathbf{f}; \tag{2.1}$$

$$\nabla \cdot \mathbf{u} = 0, \tag{2.2}$$

where $\mathbf{u} = [u, v]^T$ is the 2D velocity field of the fluid, and ρ, p and Re denote the fluid's uniform density, pressure, and Reynolds number, respectively. The additional forcing term $\mathbf{f}$ that sustains the fluid at a statistically steady state includes a term that inject energy at wavenumber $k_f = 4$ and a linear friction term with dissipation coefficient $\alpha_{\text{diss}} = 0.1$ that prevents energy accumulation at large scales.

We consider homogeneous, isotropic turbulence (HIT) in periodic boundary conditions, which serves as a simplified assumption for turbulence in planet-forming disk. Due to the direct cascade, the number of degrees of freedom required to fully resolve 2D turbulence scales linearly with the Reynolds number. In the context of planet-forming disk, where $Re \sim \mathcal{O}(10^{14})$, fully resolving all turbulent scales is computationally infeasible. Consequently, DNSs of planet-forming disk use an effective viscosity compatible with the grid resolution. In our experiment, we set $Re = 1000$, for which turbulence is fully resolved on a 2048×2048 grid.

2.2 Hybrid CNN-Solver Framework

We define $\mathcal{P}$ as one time-step of the coarse differentiable solver of equations (2.1) and (2.2). A CNN $f_{\boldsymbol{\theta}}$, with trainable parameters $\boldsymbol{\theta}$, is used to predict a correction term accounting for unresolved subgrid effects. The corrected hybrid update of the velocity field at time $t + 1$, $\mathbf{u}_{t+1}$, is computed from $\mathbf{u}_t$ and velocity fields from previous timesteps $\mathbf{u}_{t-1}, \cdots, \mathbf{u}_{t-p}$ as:

$$\mathbf{u}_{t+1} = \mathcal{P}(\mathbf{u}_t) + f_{\boldsymbol{\theta}}(\mathcal{P}(\mathbf{u}_t), \mathbf{u}_t, \mathbf{u}_{t-1}, \cdots, \mathbf{u}_{t-p}), \tag{2.3}$$

Unlike the baseline model [9] which uses only the post-solver state as input to the CNN, $\mathbf{u}_{t+1} = \mathcal{P}(\mathbf{u}_t) + f_\theta(\mathcal{P}(\mathbf{u}_t))$, our formulation (2.3) also includes $\mathbf{u}_t, \mathbf{u}_{t-1}, \cdots, \mathbf{u}_{t-p}$ in the network input. This augmented input is intended to provide temporal context through past states while retaining information about the solver dynamics through the post-solver state.

Key design choices of the framework are as follows:

Numerical Solver. We use the `jax-cfd` code, a finite-volume solver on a regular staggered grid, as the backbone $\mathcal{P}$ of our framework. A grid-based solver is chosen for consistency with most astrophysical fluid codes. The framework operates on a 64×64 grid, corresponding to a downsampling factor of 32 relative to the DNS reference simulation. Further details on the numerical schemes, see Appendix 1 of [9].

Architecture of CNN Corrector. The correction operator $f_{\boldsymbol{\theta}}$ is parameterized by a lightweight CNN with periodic padding, consistent with the solver step. In the baseline architecture [9], $f_{\boldsymbol{\theta}} : \mathbb{R}^{64\times64\times2} \to \mathbb{R}^{64\times64\times2}$ is a six-layer CNN acting on a two-channel velocity field. Each hidden layer consists of a convolution of channel size 64, batch normalization and a ReLU activation unit.

For the multi-step input variant, we stack the velocity fields from successive time-steps along the channel dimension, so that the input becomes $[\mathcal{P}(\mathbf{u}_t), \mathbf{u}_t, \mathbf{u}_{t-1}, \cdots, \mathbf{u}_{t-p}] \in \mathbb{R}^{64\times64\times2(p+2)}$. In some experiments, we keep the baseline architecture unchanged in order to isolate the effect of enlarging the temporal input alone and enable a direct comparison with [9]. In other experiments, we additionally increase the number of kernels in the first convolutional layer, using 128 kernels instead of 64, in order to increase the number of learnable parameters and provide the model with more capacity to exploit the richer multi-step input.

2.3 Solver-in-the-loop Training

Building on the *solver-in-the-loop* training strategy proposed by [14], we integrate the differentiable fluid solver directly into the training process. This allows the neural network to learn over an extended temporal horizon, strongly improving its stability and long term predictive performance.

Let $\{\mathbf{u}^{\text{ref}}_{t,i}\}^{i=1,\cdots,M}_{t=1,\cdots N-T+1}$ denote a collection of M independent high-resolution trajectories sampled at discrete times of temporal length N. We introduce a downsampling operator $\mathcal{D}$ mapping the DNS state to the coarse state space. The training loss is defined as an *a posteriori* Mean Square Error (MSE) between model predictions and the downsampled DNS states,

$$\mathcal{L}(\theta) = \sum_{i=1}^{M} \sum_{n=1}^{N-T+1} \sum_{t=n}^{n+T-1} \|\mathbf{u}^{\text{pred}}_{t,i} - \mathcal{D}(\mathbf{u}^{\text{ref}}_{t,i})\|^2_F, \tag{2.4}$$

where T is the number of unrolled time-steps seen by the neural network per optimisation step and $\{\mathbf{u}^{\text{pred}}_{t,i}\}_{t=n\ldots n+T-1}$ are recursively computed from the initial state $\mathcal{D}(\mathbf{u}^{\text{pred}}_{t,i})$ using (2.3). The tight coupling between the fluid solver and the neural network necessitates the use of a differentiable solver.

We used a set of $M = 28$ simulations for training, and 4 simulations for validation. Each simulation is initialised from a random white-noise velocity field, evolved to $t_{\text{sim}} = 40$ to reach a statistically steady state, and then run for an additional $N = 500$ timesteps. Training samples are obtained using a sliding window of length $T + 1$ along the temporal dimension, where the first timestep provides the initial conditions and the remaining timesteps serve as targets. This adds up to around 13,000 samples. Model parameters are optimised using a standard Adam optimizer with a fixed learning rate of 10^{-4} and a batch size of $B = 64$. All models are trained for 400 epochs. We note that T has to be increased progressively to maintain training stability.

3 Results and Discussion

To evaluate the framework, we performed rollouts over 3000 timesteps using 10 initial velocity fields not seen during training. We compare several models: a baseline CNN using only the post-solver state as input, trained with rollout lengths of $T = 32$ (denoted T32), same CNN trained over $T = 16$ (denoted T16) and our proposed framework, (2.3), with $p \in [0, 4]$, trained with $T = 16$ (denoted T16_p0-4). As previously mentioned, for this set of experiments we keep the baseline architecture unchanged to isolate the effect of incorporating more temporal information. Additionally, for $p = 3$ and $p = 4$, we compare the results with a larger model by increasing the number of kernels in the initial layer to 128, this configuration is denoted by an additional subscript L (T16_p0-4_L). We also report results from

downsampled high resolution DNSs as reference, and corresponding uncorrected low resolution DNSs initialised with the same velocity fields.

3.1 Mean Squared Error

As our training objective is to match the framework output to fully resolved DNS data, we compare the normalized MSEs. In Figure 1, we overlay the MSEs evaluated on training and validation sets at the last epoch 400 for the different models. We also compare MSEs across different time horizons between the models and the reference. The results are reported in Table 1.

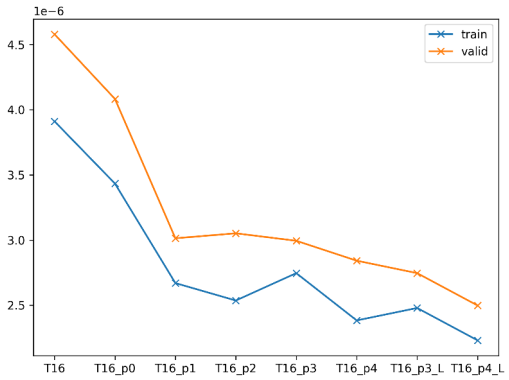


Figure 1: One-step normalized MSE loss at epoch 400 of the different models on training and validations sets.

Model	100	500	1000
T16	1.85×10^{-4}	0.0117	0.374
T32	2.14×10^{-4}	0.0159	0.48
T16_p0	2.95×10^{-4}	0.0307	0.688
T16_p1	1.40×10^{-4}	0.0073	0.216
T16_p2	2.25×10^{-4}	0.0256	0.532
T16_p3	1.44×10^{-4}	0.0078	0.254
T16_p4	1.89×10^{-4}	0.0207	0.44
T16_p3_L	1.28×10^{-4}	0.0070	0.265
T16_p4_L	1.22×10^{-4}	0.0073	0.252
No Corr	9.39×10^{-3}	0.29	1.02

Table 1: MSE at selected timesteps, averaged over the test set.

Figure 1 also shows the one-step MSE loss at the end of training for each model trained with $T = 16$. The results indicate that increasing the number of velocity states from previous timesteps leads to a lower training loss after a fixed number of epochs. All hybrid frameworks outperform the uncorrected low resolution DNS by order of magnitude. We find that, compared to T32, the T16 model already provides a strong representation of the turbulent flow. The slight degradation observed for T32 may be due to the model primarily learning mean-flow dynamics rather than dynamics at more local scales. For our proposed framework with the baseline architecture, 2.3, we find that T16_p1 yields the best MSE performance and improves upon the baseline T16 model, suggesting that incorporating temporal information enhances the model's ability to capture temporal dynamics more effectively. We further find that, with larger number of previous states p in the input, the larger model outperforms the baseline architecture, indicating that additional learnable parameters improve the representation of temporal patterns.

3.2 Turbulence statistics

Maintaining pointwise correlation in a chaotic system such as turbulence is inherently unrealistic beyond short timescales. It is, therefore, more meaningful to evaluate statistical properties of the flow. We compute the energy spectrum, structure functions, and vorticity probability density functions (PDFs) over the regime where the framework has decorrelated from the reference DNS. Figure 2 shows the corresponding logarithmic errors in the energy spectrum and the fourth-order velocity structure function relative to the reference statistics, while Figure 3 presents the vorticity PDFs at different scales.

We find that power law behaviour of the energy spectrum and velocity structure function is already well captured with relatively simple models, and the T16 models are already sufficient in reproducing statistics comparable to reference DNSs. Furthermore, for applications where the structure functions are key,

including previous timesteps as additional inputs shows clear improvements. However, when more than 2 previous states are included, the baseline architecture lacks the complexity to fully represent the additional temporal information, leading to degraded performance. A larger CNN is hence needed for further improvement. For applications to planet formation, where the vorticity PDF is decisive, our framework shows a clear improvement when 2 previous timesteps are considered (T16_p2).

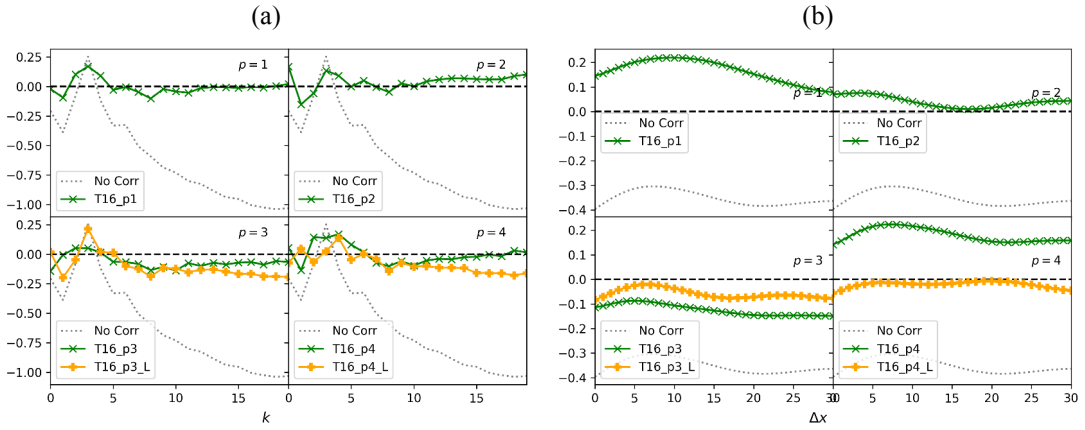


Figure 2: (a) Log difference in energy spectrum over wavenumber k and (b) log difference in fourth-order velocity structure function over cell separation Δx relative to the reference statistics averaged between 2500 and 3000 timesteps across 10 independent simulations.

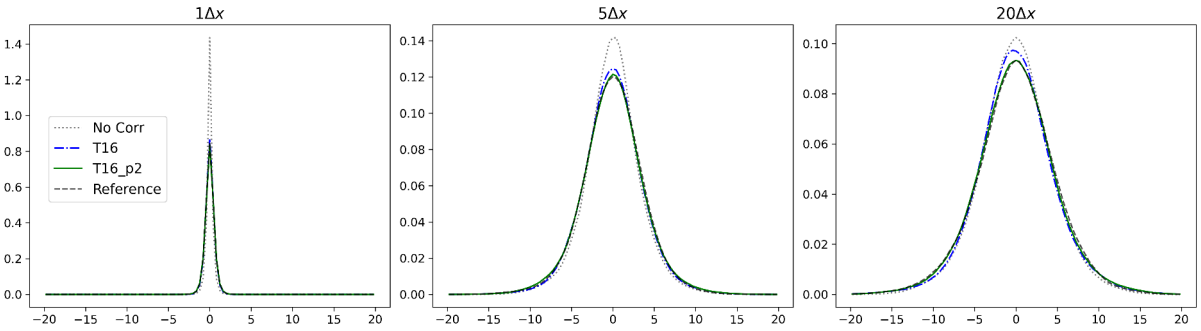


Figure 3: Vorticity PDFs at grid separation $1\Delta x, 5\Delta x$ and $20\Delta x$ respecitvely, averaging between 2500 and 3000 simulation timesteps, over 10 independent simulations.

3.3 Computation Time

Training with rollout lengths of $T = 32$ requires 12 hours over 400 epochs, whereas reducing the rollout length to $T = 16$ lowers the training time to 7 hours, while maintaining performance comparable to the T32 model. Moreover, by reducing the resolution and including the CNN, we achieve a speedup of $40\times$ compared to fully resolved DNS, despite a $4\times$ increase in computing time compared to the uncorrected low resolution simulation.

4 Conclusion

In conclusion, we build on prior work leveraging differentiable physics to develop a hybrid framework that combines a grid-based numerical solver with a convolutional neural network to model turbulence accurately at reduced computation cost. By providing the network with velocity fields from previous

timesteps, we incorporate additional temporal information that improves training efficiency, while reproducing statistical properties of turbulence comparable to reference data. We further show that the rollout length during training can be reduced by a factor of two without significant degradation in performance, substantially lowering the overall training cost. However, our results also suggest that larger network architectures may be required to fully exploit the increased complexity introduced by additional temporal inputs. These findings highlight the potential of hybrid solver-CNN frameworks for developing turbulence models in astrophysical fluid dynamics. Future work will investigate the dynamics of particles evolving within the predicted turbulent flow.

Acknowledgement

This work was supported by the French government through the France 2030 investment plan managed by the National Research Agency (ANR), as part of the Initiative of Excellence Université Côte d'Azur under reference number ANR-15-IDEX-01, and as part of the PEPR Origins under reference number ANR-22-EXOR-0001. This project was provided with computing HPC/AI and storage resources by GENCI at IDRIS thanks to the grant 2024-AD010415910 on the supercomputer Jean Zay's V100/A100 partition.

References

[1] Kevin M. Flaherty et al. "Weak Turbulence in the HD 163296 Protoplanetary Disk Revealed by ALMA CO Observations". In: *ApJ* 813.2, 99 (Nov. 2015), p. 99.

[2] Jinshi Sai et al. "Nonthermal Velocity Dispersion in the Outer Disk of HL Tau". In: *arXiv e-prints*, arXiv:2603.29147 (Mar. 2026).

[3] Steven A. Balbus and John F. Hawley. "Instability, turbulence, and enhanced transport in accretion disks". In: *Reviews of Modern Physics* 70.1 (Jan. 1998), pp. 1–53.

[4] Sebastien Fromang and John Papaloizou. "Dust settling in local simulations of turbulent protoplanetary disks". In: *A&A* 452.3 (June 2006), pp. 751–762.

[5] Andrew Youdin and Anders Johansen. "Protoplanetary Disk Turbulence Driven by the Streaming Instability: Linear Evolution and Numerical Methods". In: *ApJ* 662.1 (June 2007), pp. 613–626.

[6] John F. Hawley, Charles F. Gammie, and Steven A. Balbus. "Local Three-dimensional Magnetohydrodynamic Simulations of Accretion Disks". In: *ApJ* 440 (Feb. 1995), p. 742.

[7] Heloise Meheut et al. "Angular momentum transport and large eddy simulations in magnetorotational turbulence: the small Pm limit". In: *A&A* 579, A117 (July 2015), A117.

[8] Nils Thuerey et al. *Physics-based Deep Learning*. 2021. URL: `https://physicsbaseddeeplearning.org`.

[9] Dmitrii Kochkov et al. "Machine learning-accelerated computational fluid dynamics". In: *Proceedings of the National Academy of Science* 118.21 (May 2021).

[10] Björn List, Li-Wei Chen, and Nils Thuerey. "Learned turbulence modelling with differentiable fluid solvers: physics-based loss functions and optimisation horizons". In: *Journal of Fluid Mechanics* 949, A25 (Oct. 2022), A25.

[11] Hugo Frezat et al. "A Posteriori Learning for Quasi-Geostrophic Turbulence Parametrization". In: *Journal of Advances in Modeling Earth Systems* 14.11 (Nov. 2022).

[12] André Freitas et al. "Solver-in-the-loop approach to closure of shell models of turbulence". In: *Physical Review Fluids* 10.4 (Apr. 2025).

[13] Oded Regev and Orkan M. Umurhan. "On the viability of the shearing box approximation for numerical studies of MHD turbulence in accretion disks". In: *A&A* 481.1 (Apr. 2008), pp. 21–32.

[14] Kiwon Um et al. "Solver-in-the-Loop: Learning from Differentiable Physics to Interact with Iterative PDE-Solvers". In: *Advances in Neural Information Processing Systems* (2020).